

CIA III – Simulator Project

4:1 Multiplexer & 3:8 Decoder Simulator

Submitted by	Agamveer	Register no.: 2643006
Programme	BCA Honours, Christ University, Yeshwanthpur	
Assessment	CIA III – Simulator Project (20 marks)	

1. Chosen topic & objectives

This project implements two combinational logic circuits from Unit 2 — Digital Design Fundamentals & Boolean Algebra: a **4:1 multiplexer (MUX)** and a **3:8 decoder**. Both are built as a single interactive web simulator, along with this supporting report.

Objectives:

- Design and simplify the Boolean logic for a 4:1 MUX and a 3:8 decoder from first principles.
- Implement both circuits as an interactive, browser-based simulator with live truth tables.
- Verify correct behaviour by exercising every input combination and logging the results as test cases.
- Document the gate-level design, algorithm, and test evidence in this report.

2. Topic overview

A **multiplexer** is a combinational circuit that selects one of several input lines and routes it to a single output, based on the value of a set of select lines. A 4:1 MUX has four data inputs (D0–D3), two select lines (S1, S0), and one output (Y) — the select lines form a 2-bit address that picks which data input reaches Y.

A **decoder** does the opposite job: it takes a binary address and activates exactly one of many output lines. A 3:8 decoder reads a 3-bit address (A2, A1, A0) and drives exactly one of eight outputs (Y0–Y7) high, provided an enable line (E) is asserted; with E low, every output stays low regardless of the address.

3. System design

3.1 4:1 multiplexer

Each data input is ANDed with the minterm of the select lines that corresponds to its address; the four AND outputs are then combined with a single OR gate to produce Y. Only one AND gate can be active for any given select combination, so only one data input ever reaches the output.

$$Y = D0 \cdot S1' \cdot S0' + D1 \cdot S1' \cdot S0 + D2 \cdot S1 \cdot S0' + D3 \cdot S1 \cdot S0$$

S1	S0	Selected input	Y
0	0	D0	D0
0	1	D1	D1

1	0	D2	D2
1	1	D3	D3

Table 1: 4:1 MUX truth table.

4:1 multiplexer — gate level

$$Y = D0S1'S0' + D1S1'S0 + D2S1S0' + D3S1S0$$

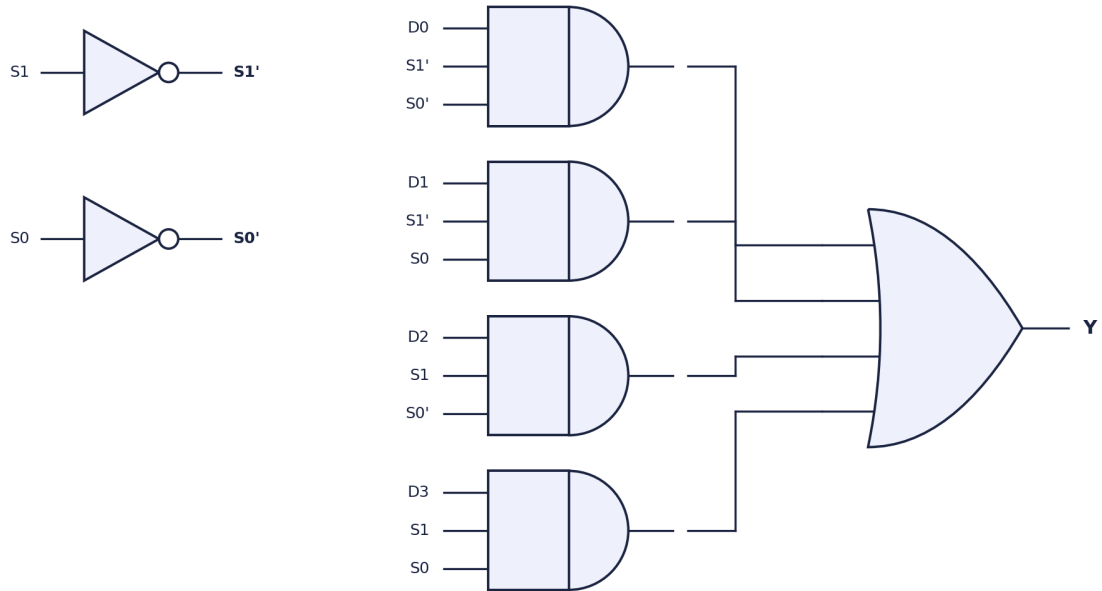


Figure 1: Gate-level circuit — two inverters generate the complemented select lines; four 3-input AND gates each recognise one address; a 4-input OR gate merges them into Y .

3.2 3:8 decoder

Each output Y_i is produced by its own AND gate, which takes the enable line E together with the literal combination of A_2, A_1, A_0 that spells out address i in binary (using the true or complemented form of each address bit as required). Exactly one AND gate can be satisfied at a time, so exactly one output goes high when $E = 1$.

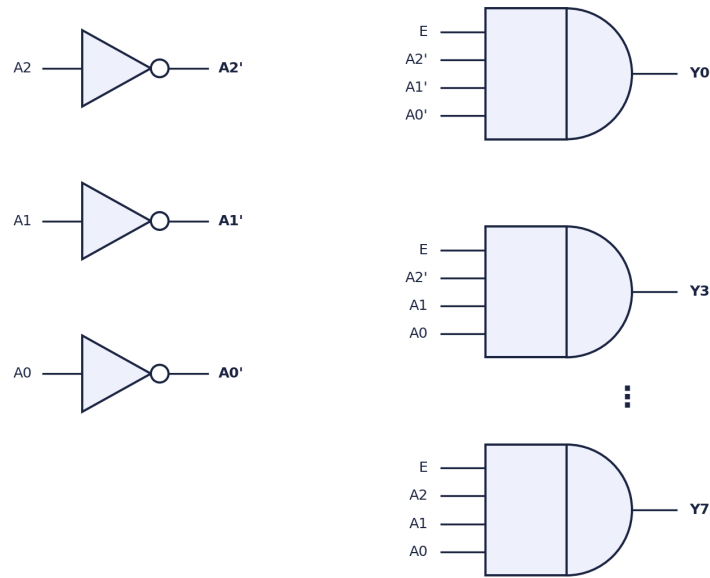
$Y_i = E \cdot m_i(A_2, A_1, A_0)$ where m_i is the i -th minterm of the 3-bit address

E	A2	A1	A0	Active output
1	0	0	0	Y0
1	0	0	1	Y1
1	0	1	0	Y2
1	0	1	1	Y3
1	1	0	0	Y4
1	1	0	1	Y5
1	1	1	0	Y6
1	1	1	1	Y7
0	X	X	X	none (all outputs low)

Table 2: 3:8 decoder truth table (X = don't care, since the address has no effect while $E = 0$).

3:8 decoder — gate level (representative outputs)

$Y_i = E \cdot m_i(A_2, A_1, A_0)$ — one 4-input AND gate per output line



Y_1, Y_2, Y_4, Y_5, Y_6 follow the same pattern: a 4-input AND gate gated by E and the literal combination of A_2, A_1, A_0 for that address.

Figure 2: Gate-level circuit — three inverters generate the complemented address lines; each output has its own 4-input AND gate. Y_0, Y_3 and Y_7 are shown in full; the remaining five outputs (Y_1, Y_2, Y_4, Y_5, Y_6) are wired identically, each to its own address minterm.

4. Implementation

Both circuits are implemented as a single interactive web page (HTML, CSS and JavaScript, no external dependencies). Each input is a toggle switch; the schematic wires light up to show which signals are currently high, the truth table highlights the row matching the current inputs, and the displayed Boolean equation updates to show exactly which term is active. A test-case log lets each input combination be captured and exported for this report.

5. Test cases

The full input space was exercised on the live simulator: all 4 select combinations for the MUX and all 8 addresses (plus the disabled case) for the decoder, matching Table 1 and Table 2 above.

5.1 MUX test cases

- $S_1S_0 = 00$ ($Y = D_0$)
- $S_1S_0 = 01$ ($Y = D_1$)
- $S_1S_0 = 10$ ($Y = D_2$)
- $S_1S_0 = 11$ ($Y = D_3$)

5.2 Decoder test cases

All 8 enabled addresses (Y_0 – Y_7) plus one disabled case ($E = 0$) were verified against the simulator's test-case log, matching Table 2 above.

6. Conclusion

The simulator correctly reproduces the truth table of both a 4:1 multiplexer and a 3:8 decoder across every input combination, confirming that the gate-level designs in Section 3 are implemented correctly. The project meets all three submission requirements: a working simulator link, this supporting report, and a logged set of test cases covering the full input space of both circuits.